SMART CHEF & PANTRY AI

Flask Backend Architecture

Versiyon: 1.0
Backend: Python / Flask
Database: PostgreSQL
ORM: SQLAlchemy
Migration: Flask-Migrate / Alembic
API: REST / JSON
Authentication: Social Login + JWT

---

1. Amaç

Bu dokümanın amacı Smart Chef & Pantry AI backend uygulamasının Flask tarafındaki mimarisini kesinleştirmektir.

Backend aşağıdaki sorumlulukları yerine getirecektir:

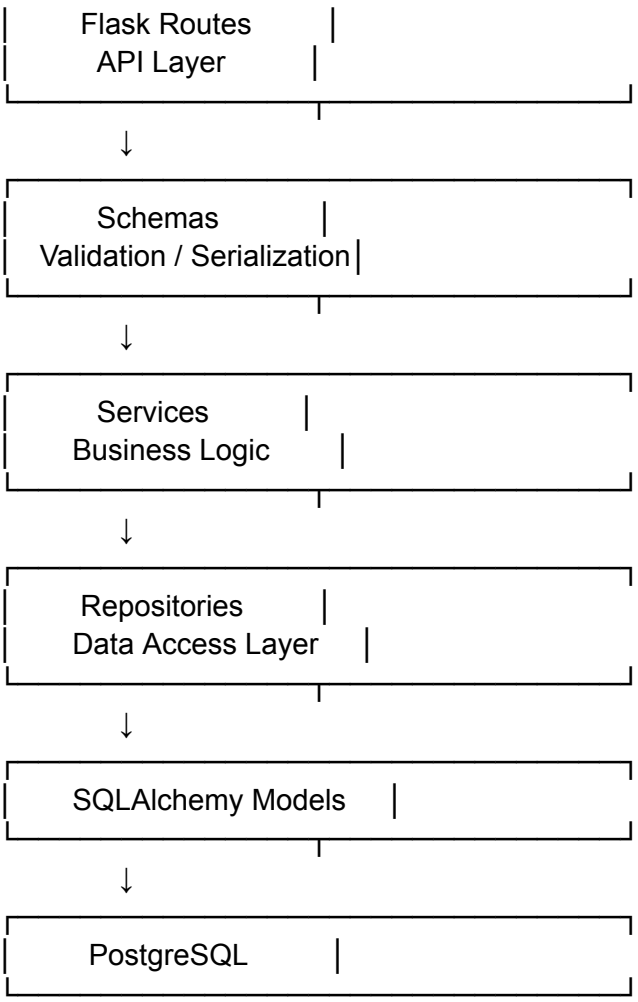
- REST API sağlamak
- Kullanıcı authentication işlemlerini yönetmek
- Recipe ve ingredient verilerini yönetmek
- Recipe matching işlemini gerçekleştirmek
- Favorite işlemlerini yönetmek
- Shopping cart işlemlerini yönetmek
- PostgreSQL database erişimini yönetmek
- Admin tarafından curated recipe dataset'inin yönetilmesini sağlamak
- Flutter'a standart JSON response döndürmek

---

2. Genel Backend Mimarisi

Ana yapı:





---

3. Flask Klasör Yapısı

Önerilen backend yapısı:

```
backend/
│
├── app/
│   ├── __init__.py
│   │
│   ├── config/
│   │   ├── __init__.py
│   │   └── settings.py
│   │
│   ├── extensions/
│   │   ├── __init__.py
│   │   ├── database.py
│   │   └── jwt.py
```

```
│   │
│   ├── models/
│   │   ├── __init__.py
│   │   ├── user.py
│   │   ├── ingredient.py
│   │   ├── recipe.py
│   │   ├── recipe_ingredient.py
│   │   ├── favorite.py
│   │   └── cart_item.py
│   │
│   ├── schemas/
│   │   ├── __init__.py
│   │   ├── auth.py
│   │   ├── ingredient.py
│   │   ├── recipe.py
│   │   ├── favorite.py
│   │   ├── cart.py
│   │   └── common.py
│   │
│   ├── routes/
│   │   ├── __init__.py
│   │   ├── auth_routes.py
│   │   ├── ingredient_routes.py
│   │   ├── recipe_routes.py
│   │   ├── favorite_routes.py
│   │   ├── cart_routes.py
│   │   └── admin_routes.py
│   │
│   ├── services/
│   │   ├── auth_service.py
│   │   ├── ingredient_service.py
│   │   ├── recipe_service.py
│   │   ├── favorite_service.py
│   │   └── cart_service.py
│   │
│   ├── repositories/
│   │   ├── user_repository.py
│   │   ├── ingredient_repository.py
│   │   ├── recipe_repository.py
│   │   ├── favorite_repository.py
│   │   └── cart_repository.py
│   │
│   ├── ai/
│   │   ├── __init__.py
│   │   ├── ingredient_normalizer.py
│   │   ├── recipe_matcher.py
│   │   └── recommendation_engine.py
│   │
```

```
│   ├── utils/
│   │   ├── response.py
│   │   ├── validators.py
│   │   └── decorators.py
│   │
│   └── errors/
│       ├── __init__.py
│       ├── exceptions.py
│       └── handlers.py
│
├── migrations/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── fixtures/
│
├── .env
├── .env.example
├── requirements.txt
├── config.py
└── run.py
```

---

4. Application Factory

Flask uygulaması application factory yaklaşımıyla oluşturulacaktır.

Temel yapı:

```
run.py
  ↓
create_app()
  ↓
Flask Application
  ↓
Extensions
  ↓
Blueprints
```

Örneğin:

```
def create_app():
    app = Flask(__name__)

    register_extensions(app)
```

```
    register_blueprints(app)
    register_error_handlers(app)

    return app
```

Böylece test ortamı, development ortamı ve production ortamı birbirinden ayrılabilir.

---

5. Config Yapısı

Configuration:

```
app/config/
└── settings.py
```

Ortam değişkenleri:

```
DATABASE_URL
JWT_SECRET_KEY
GOOGLE_CLIENT_ID
GOOGLE_CLIENT_SECRET
FLASK_ENV
```

gibi bilgiler `.env` üzerinden alınacaktır.

Secret bilgiler GitHub repository'sine gönderilmeyecektir.

`.gitignore`:

```
.env
__pycache__/
.pytest_cache/
```

---

6. Extensions

Ortak Flask extension'ları:

```
app/extensions/
├── database.py
└── jwt.py
```

Database:

SQLAlchemy

Authentication:

JWT

Migration:

Flask-Migrate / Alembic

şeklinde planlanmaktadır.

---

7. Database Layer

SQLAlchemy model'leri:

```
app/models/
├── user.py
├── ingredient.py
├── recipe.py
├── recipe_ingredient.py
├── favorite.py
└── cart_item.py
```

Bu modeller database tablolarını temsil eder.

Örneğin:

```
Recipe
  │
  ├── id
  ├── title
  ├── description
  ├── instructions
  ├── image_url
  ├── source_url
  ├── youtube_url
  └── category
```

---

8. Repository Layer

Repository'lerin görevi database erişimidir.

Örneğin:

RecipeRepository

şunları yapabilir:

get_by_id()
get_all()
find_by_category()
find_by_ingredients()
create()
update()
delete()

Ancak recommendation business logic'i repository içinde olmayacaktır.

Yanlış:

RecipeRepository
↓
match_percentage hesapla

Doğru:

RecipeService
↓
RecipeMatchingService
↓
RecipeRepository

---

9. Service Layer

Service katmanı business logic'in ana merkezidir.

```
app/services/
├── auth_service.py
├── ingredient_service.py
├── recipe_service.py
├── favorite_service.py
└── cart_service.py
```

Her service belirli bir domain sorumluluğunu üstlenecektir.

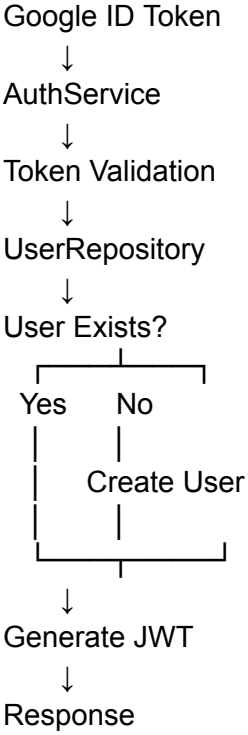
---

10. AuthService

`AuthService` sorumlulukları:

1. Social token doğrulama
2. Provider bilgisini kontrol etme
3. Kullanıcıyı bulma
4. Kullanıcı yoksa oluşturma
5. Backend JWT üretme
6. User response oluşturma

Akış:



---

11. IngredientService

Sorumlulukları:

- Ingredient listesini getirmek
- Search yapmak
- Category filter uygulamak
- Ingredient validation
- Admin ingredient CRUD işlemleri

Örnek:

GET /api/ingredients
↓
IngredientRoute
↓
IngredientService
↓
IngredientRepository
↓
PostgreSQL

---

12. RecipeService

RecipeService:

- Recipe detail
- Recipe list
- Recommendation işleminin orchestration'ı
- Recipe validation
- Admin recipe CRUD

Önemli nokta:

RecipeService recommendation algoritmasının tüm detaylarını kendisi taşımayacaktır.

Bunun yerine:

RecipeService
↓
RecommendationEngine
↓
RecipeMatcher

yapısı kullanılacaktır.

---

13. FavoriteService

Sorumlulukları:

- Favorite ekleme
- Favorite silme
- Favorite listeleme
- Duplicate kontrolü
- Kullanıcı sahipliği kontrolü

Akış:

POST /api/favorites
↓
FavoriteRoute
↓
FavoriteService
↓
FavoriteRepository
↓
Database

Duplicate:

User A
+
Recipe X
↓
Favorite exists?
↓
Yes
↓
409 FAVORITE_ALREADY_EXISTS

---

14. CartService

Cart business logic burada tutulacaktır.

Sorumlulukları:

- Cart listeleme
- Ingredient ekleme
- Quantity güncelleme
- Cart item silme
- Cart temizleme
- Aynı ingredient'ların merge edilmesi

Önemli örnek:

Mevcut:
Domates = 2 adet

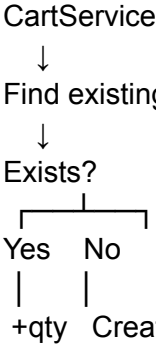
Yeni:
Domates = 3 adet

CartService:
2 + 3

Sonuç:
Domates = 5 adet

Repository sadece database işlemini yapacaktır.

Merge kararı:



---

15. AI Katmanı

AI/matching logic:

```
app/ai/
├── ingredient_normalizer.py
├── recipe_matcher.py
└── recommendation_engine.py
```

Bu klasörün MVP'de gerçek bir deep learning modelinden oluşması gerekmemektedir.

İlk aşamada deterministic recommendation sistemi kullanılacaktır.

---

16. Ingredient Normalizer

Görevi kullanıcı ve recipe ingredient isimlerini karşılaştırılabilir hale getirmektir.

Örneğin:

" Domates "
"domates"
"DOMATES"

gibi değerler:

domates

şeklinde normalize edilebilir.

Ancak:

domates
çeri domates
domates sosu

otomatik olarak aynı ingredient kabul edilmemelidir.

Bu ayrım semantic matching aşamasında ayrıca ele alınacaktır.

---

17. RecipeMatcher

RecipeMatcher temel ingredient matching işlemini yapacaktır.

Input:

User Ingredients
+
Recipe Ingredients

Output:

matched ingredients
missing ingredients
match percentage

Örnek:

User:
Tavuk
Domates
Soğan

Recipe:
Tavuk
Domates
Soğan
Biber

Matched = 3
Required = 4

Score = 75%

---

18. RecommendationEngine

RecommendationEngine RecipeMatcher'ın çıktısını kullanarak tarifleri sıralayabilir.

MVP:

Match Percentage
↓
Descending Sort
↓
Recipe Results

Örneğin:

Recipe A → 100%
Recipe B → 80%
Recipe C → 75%
Recipe D → 50%

İlk sürümde temel sıralama yeterlidir.

---

19. Recommendation Engine'in Gelecekte Genişletilmesi

MVP sonrasında:

Match Score
+
Ingredient Coverage
+
Category Preference
+
Recipe Popularity
+
User History

gibi faktörler eklenebilir.

Daha ileri aşamada:

Content-Based Recommendation

veya:

Machine Learning Recommendation

eklenebilir.

LLM ise:

"Evde tavuk, patates ve yoğurt var."

gibi doğal dil girdilerinden ingredient çıkarma amacıyla kullanılabilir.

LLM doğrudan database'den tarif seçen kontrolsüz bir sistem haline getirilmemelidir.

---

20. Route Layer

Routes API'nin dış dünyaya açılan noktasıdır.

Örneğin:

```
app/routes/
├── auth_routes.py
├── ingredient_routes.py
├── recipe_routes.py
├── favorite_routes.py
├── cart_routes.py
└── admin_routes.py
```

Route'un görevi:

1. Request almak
2. Authentication kontrolünü tetiklemek
3. Schema validation yapmak
4. Service çağırmak
5. Response döndürmek

Route içerisinde business logic yazılmamalıdır.

---

21. Route Örneği

Yanlış yaklaşım:

```
@app.post("/recipes/recommend")
def recommend():
    ingredients = request.json["ingredient_ids"]

    recipes = Recipe.query.all()

    # matching logic
    # score calculation
    # sorting
    # database operations

    return ...
```

Bu yaklaşım proje büyüdüğünde route'ları kontrol edilemez hale getirir.

Doğru yaklaşım:

Route

↓
Schema
↓
RecipeService
↓
RecommendationEngine
↓
Repository

---

22. Schema Layer

Schema katmanı API input/output validation için kullanılacaktır.

Örneğin:

app/schemas/recipe.py

içerisinde:

RecipeRecommendationSchema
RecipeResponseSchema
RecipeIngredientSchema

gibi yapılar bulunabilir.

Request:

```
{
  "ingredient_ids": [
    "uuid-1",
    "uuid-2"
  ]
}
```

önce validation'dan geçer.

Geçersizse service çalıştırılmaz.

---

23. Error Handling

Global error handler kullanılacaktır.

Örneğin:

ValidationError
AuthenticationError
NotFoundError
ConflictError
ForbiddenError
DatabaseError

gibi exception'lar ortak response formatına dönüştürülecektir.

Örnek:

```
{
  "success": false,
  "error": {
    "code": "RECIPE_NOT_FOUND",
    "message": "Recipe not found."
  }
}
```

---

24. Authentication Middleware

Protected endpoint'lerde JWT kontrolü yapılacaktır.

Örneğin:

GET /api/cart

için:

Request
↓
JWT Validation
↓
User ID
↓
CartService

kullanılacaktır.

Service'e mümkün olduğunca:

current_user_id

gibi doğrulanmış identity bilgisi aktarılmalıdır.

Client'ın gönderdiği:

user_id

güvenilir kabul edilmemelidir.

---

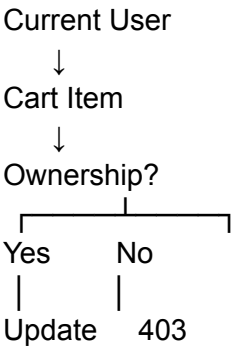
25. Kullanıcı Sahipliği Kontrolü

Örneğin Cart Item:

cart_item_id = X

kullanıcının gerçekten kendisine ait olmayan bir item'ı değiştirmesine izin verilmemelidir.

Kontrol:



Aynı kontrol:

- Favorites
- Cart
- Profile

gibi kullanıcıya özel kaynaklarda uygulanacaktır.

---

26. Admin Authorization

Admin endpoint'leri:

/api/admin/*

olarak ayrılacaktır.

Akış:

JWT
↓
User
↓
Admin Authorization
↓
Admin Route

Admin olmayan kullanıcı:

403 FORBIDDEN

alacaktır.

---

27. Recipe Recommendation Tam Akışı

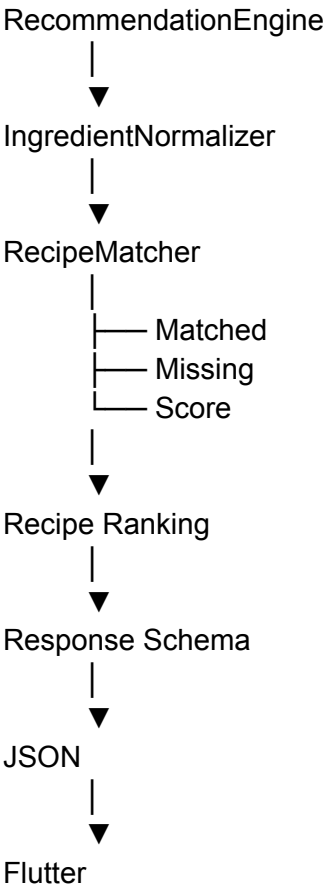
Bu projenin backend açısından en önemli akışı:

POST /api/recipes/recommend

Recipe Route

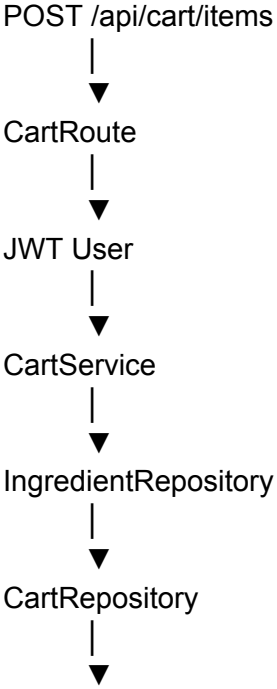
Request Schema Validation

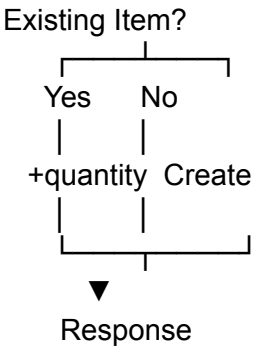
RecipeService

---

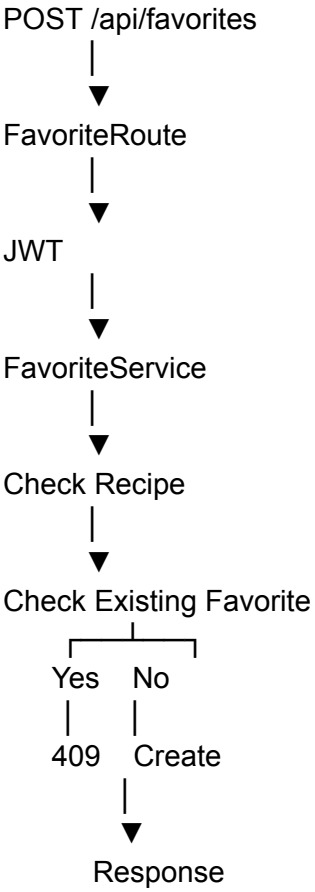
28. Cart İşlem Akışı





---

29. Favorite İşlem Akışı



---

30. Database Transaction Yaklaşımı

Birden fazla database işleminin birlikte başarılı olması gereken durumlarda transaction kullanılacaktır.

Örneğin recipe oluşturma:

Create Recipe
↓
Create Recipe Ingredients
↓
Commit

Eğer ingredient bağlantılarından biri başarısız olursa:

Rollback

yapılmalıdır.

Böylece yarım recipe kayıtlarının oluşması önlenir.

---

31. Migration Yapısı

Database schema değişiklikleri doğrudan production database üzerinde elle yapılmayacaktır.

Migration:

Model Change
↓
Migration
↓
Review
↓
Upgrade Database

Örneğin:

001_initial_schema
002_add_recipe_category
003_add_display_quantity

gibi migration geçmişi tutulabilir.

---

32. Logging

Backend'de en azından aşağıdaki olaylar loglanmalıdır:

Request
Authentication failure
Validation failure
Database error
Unexpected exception

Ancak:

JWT
Social access token
Password
Secret key

gibi hassas bilgiler loglanmamalıdır.

---

33. Environment Yapısı

Development:

.env

Production:

Production environment variables

kullanacaktır.

Örnek:

DATABASE_URL=...
JWT_SECRET_KEY=...
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...

`.env.example`:

DATABASE_URL=
JWT_SECRET_KEY=
GOOGLE_CLIENT_ID=

GOOGLE_CLIENT_SECRET=

şeklinde tutulabilir.

Gerçek değerler repository'ye eklenmeyecektir.

---

34. Testing Yapısı

Backend testleri:

tests/
├── unit/
├── integration/
└── fixtures/

Unit Tests

Özellikle:

IngredientNormalizer
RecipeMatcher
RecommendationEngine
CartService
FavoriteService

test edilmelidir.

Integration Tests

Örneğin:

POST /api/recipes/recommend

endpoint'inin:

Route
→ Service
→ Repository
→ Database

akışının doğru çalıştığı kontrol edilir.

---

35. MVP'de Test Edilecek Kritik Senaryolar

Recipe Matching

3/4 required → 75%
4/4 required → 100%
0/4 required → 0%
optional ingredient missing → required score unaffected

Cart

2 tomatoes + 3 tomatoes = 5 tomatoes

Favorite

First add → success
Second add → 409
Remove → success

Authentication

Valid token → authenticated
Invalid token → 401
Expired token → 401

Authorization

User A → User B cart item
→ 403

---

36. Backend Bağımlılıkları

Planlanan temel Python paketleri:

Flask

Flask-SQLAlchemy
Flask-Migrate
Flask-JWT-Extended
psycopg2-binary
python-dotenv
pytest

Social authentication ve schema validation için kullanılacak paketler implementasyon aşamasında seçilecektir.

Burada özellikle paketlerin güncel API'lerine göre kurulum yapılması gerekmektedir; dokümantasyon üzerinden sürüm uyumluluğu kontrol edilmeden rastgele paket sürümü sabitlenmeyecektir.

---

37. Backend Sorumluluk Matrisi

Katman	Sorumluluk
Route	HTTP request/response
Schema	Validation / serialization
Service	Business logic
Repository	Database access
Model	Database representation
AI	Matching / recommendation
Error Handler	Standard error response
Middleware/Decorator	Authentication / authorization

---

38. En Kritik Mimari Kurallar

Backend geliştirilirken şu kurallar ihlal edilmemelidir:

Kural 1

Route içerisinde database sorgusu yazılmayacaktır.

Kural 2

Route içerisinde recommendation algoritması yazılmayacaktır.

Kural 3

Repository içerisinde business logic yazılmayacaktır.

Kural 4

Flutter'dan gelen `user_id` authentication için güvenilir kabul edilmeyecektir.

Kural 5

JWT ile doğrulanmış kullanıcı kimliği kullanılacaktır.

Kural 6

Recipe matching Flutter'a taşınmayacaktır.

Kural 7

Secret bilgiler GitHub'a gönderilmeyecektir.

Kural 8

API response formatı dokümandaki contract'a uygun olacaktır.

Kural 9

Database schema değişiklikleri migration üzerinden yapılacaktır.

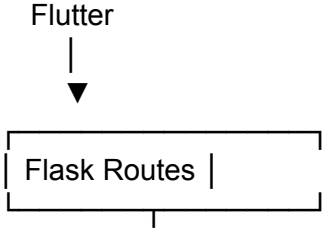
Kural 10

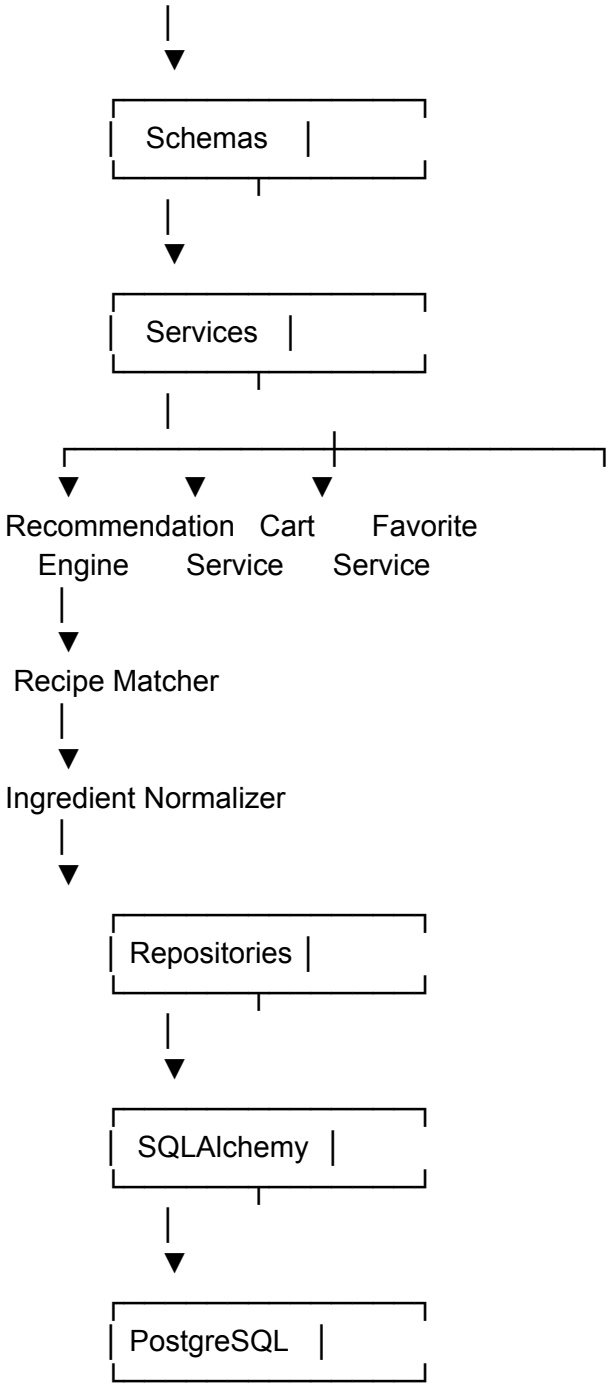
MVP'de AI adına gereksiz deep learning modeli eklenmeyecektir.

---

39. Tam Backend Veri Akışı

Smart Chef & Pantry AI'ın backend mimarisi:





---

40. Sonuç

Smart Chef & Pantry AI backend'i; route, validation, business logic ve database erişiminin birbirinden ayrıldığı katmanlı bir Flask mimarisiyle geliştirilecektir.

Ana yapı:

Routes
↓
Schemas
↓
Services
↓
AI / Business Logic
↓
Repositories
↓
SQLAlchemy
↓
PostgreSQL

Bu yapı MVP için yeterince düzenli olmakla birlikte gereksiz enterprise karmaşıklığı da oluşturmamaktadır.

İleride recommendation sisteminin geliştirilmesi gerektiğinde mevcut API ve Flutter katmanı korunarak yalnızca AI/recommendation katmanı geliştirilebilir.

---

41. Sonraki Aşama

Mimari artık iki tarafta da tanımlanmıştır:

Flutter Clean Architecture
+
Flask Backend Architecture
+
REST API Contract
+
Database Design

Bundan sonraki teknik doküman:

Doküman 09 — Recipe Matching & Recommendation Algorithm Specification

olacaktır.

Bu dokümanda artık projenin gerçekten "AI destekli" kısmını tasarlayacağız:

Ingredient Normalization
↓
Exact Matching
↓
Required / Optional ayrımı
↓
Match Percentage
↓
Recipe Ranking
↓
Weighted Recommendation
↓
İleride ML / NLP entegrasyonu

Burada özellikle algoritmanın matematiksel olarak nasıl çalışacağını ve hangi durumda hangi tarifin neden hangi skorla döneceğini kesinleştireceğiz.